Tabular Reinforcement Learning in the Stochastic Windy Gridworld

Juan Sebastián Meléndez Granados [1] **Teammate Name** [1] **Teammate Name** [1]

Abstract

This work studies several reinforcement learning algorithms in the Stochastic Windy Gridworld environment. Dynamic programming is first used to compute the optimal solution, which serves as a baseline for model-free methods. We analyze the effects of exploration strategies, compare SARSA and Q-learning, and evaluate different backup depths using n-step and Monte Carlo methods. The results illustrate how exploration, update rules, and backup depth influence learning performance in stochastic environments.

1. Introduction

Reinforcement learning (RL) studies how agents learn to make sequential decisions through interaction with an environment. Instead of receiving explicit supervision, an agent improves its behavior through trial and error by observing the rewards associated with its actions.

A common benchmark for studying RL algorithms is the Windy Gridworld introduced by (Sutton & Barto, 2018). In this environment an agent must navigate a grid while being affected by vertical wind disturbances. The stochastic version of this problem introduces uncertainty in the transitions, making it a useful testbed for evaluating learning algorithms.

In this work we study several reinforcement learning methods in the Stochastic Windy Gridworld. Dynamic programming is first used to compute the optimal solution, which serves as a baseline for model-free methods. We then analyze the impact of exploration strategies, compare SARSA and Q-learning, and study the effect of backup depth using n-step and Monte Carlo methods.

[1]Leiden University, Leiden, The Netherlands. Correspondence to: Juan Sebastián Meléndez Granados <your-email-here>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

2. Theory

Reinforcement learning problems are commonly modeled as a Markov Decision Process (MDP) defined by states S, actions A, transition probabilities $p(s'|s,a)$, and rewards $r(s,a,s')$. The goal of the agent is to learn a policy $\pi(a|s)$ that maximizes the expected cumulative reward.

A central quantity in many RL algorithms is the action-value function $Q(s,a)$, which estimates the expected return obtained by taking action a in state s and following a policy thereafter.

SARSA SARSA is an on-policy temporal-difference (TD) method that updates value estimates using the action actually selected by the current policy:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma Q(s',a') - Q(s,a)\right] \quad (1)$$

Q-learning Q-learning is an off-policy TD algorithm that updates toward the greedy value of the next state:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]. \quad (2)$$

This update allows the agent to learn the optimal action-value function independently of the exploration policy used during learning.

Monte Carlo and n-step Methods Monte Carlo methods estimate value functions using the total return of complete episodes. In contrast, n-step methods use the rewards collected over the next n steps before bootstrapping from the value function, providing a compromise between one-step TD updates and full Monte Carlo returns. This allows the algorithm to balance the bias–variance trade-off between short TD updates and long trajectory returns.

3. Experiments

3.1. Environment

All experiments were conducted in the Stochastic Windy Gridworld (Sutton & Barto, 2018), a 10×7 grid with 70

states. The agent starts at $(0, 3)$ and aims to reach the goal at $(7, 3)$.

At each timestep the agent selects one of four actions: up, right, down, or left. After the action is applied, a vertical wind may push the agent upward depending on the column. The wind strengths per column are $(0, 0, 0, 1, 1, 1, 2, 2, 1, 0)$ and are applied stochastically with probability 0.9.

Each non-terminal step yields a reward of -1, encouraging short paths, while reaching the goal yields $+100$ and terminates the episode.

3.2. Dynamic Programming Baseline

We first solved the Stochastic Windy Gridworld using tabular Q-value iteration, which provides an optimal reference policy for the later reinforcement learning experiments. Since the transition model $p(s'|s, a)$ and reward function $r(s, a, s')$ are available in the environment implementation, the action-value estimates can be updated directly using the Bellman optimality equation (Plaat, 2022):

$$Q(s, a) \leftarrow \sum_{s'} p(s'|s, a) \left[r(s, a, s') + \gamma \max_{a'} Q(s', a') \right]. \tag{3}$$

All state–action values were initialized to zero and updated iteratively until convergence using a threshold $\theta = 0.001$. Appendix 5 illustrates the progression of Q-value iteration from initialization to convergence.

For the original goal located at $(7, 3)$, the converged optimal value at the start state $(0, 3)$ is

$$V^*(s = 3) = \max_a Q^*(3, a) \approx 83.7.$$

Since the terminal reward is $+100$ and each step incurs a penalty of -1, this value corresponds to an expected trajectory length of approximately

$$T \approx 100 - 83.7 = 16.3$$

steps to reach the goal under the optimal policy.

When the goal is moved from $(7, 3)$ to $(6, 2)$, the optimal policy changes because the target lies in a wind-sensitive column. In this case the optimal start-state value decreases to approximately $V^*(s = 3) \approx 64.9$, indicating a more difficult trajectory.

3.3. Exploration Strategies

To study exploration, we implemented Q-learning with two action-selection strategies: ε-greedy and softmax exploration. The Q-table was initialized with $Q(s, a) = 0$ for all state–action pairs and updated using the rule in Eq. 2.

ε-greedy exploration The ε-greedy strategy selects the greedy action with probability $1 - \varepsilon$ and a random action with probability ε:

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A|}, & \text{if } a = \arg\max_b Q(s, b), \\ \frac{\varepsilon}{|A|}, & \text{otherwise.} \end{cases} \tag{4}$$

Softmax exploration Softmax exploration assigns action probabilities according to the Q-values:

$$\pi(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_b e^{Q(s,b)/\tau}}, \tag{5}$$

where τ controls the exploration level.

Experimental setup We evaluated ε-greedy with $\varepsilon \in \{0.03, 0.1, 0.3\}$ and softmax with $\tau \in \{0.01, 0.1, 1.0\}$. Experiments used $\alpha = 0.1$, $\gamma = 1.0$, and ε-greedy evaluation every 1000 steps over $50\,001$ timesteps, averaged across 20 repetitions.

Results Figure 1 shows the learning curves. Returns improve as the agent discovers more efficient trajectories.

Among the tested methods, ε-greedy with $\varepsilon = 0.1$ achieves the best balance between exploration and exploitation. Smaller exploration slows learning, while large softmax temperatures maintain excessive exploration. The learned policies approach but do not fully match the dynamic programming baseline because Q-learning estimates values from sampled experience.

Multiple-goal setting We introduced a second goal at $(3, 2)$ with reward $+5$ while keeping the original goal at $(7, 3)$ with reward $+100$.

Because the nearby reward can be reached in only a few steps, the agent often converges to this suboptimal goal instead of discovering the distant optimal reward. This experiment illustrates how simple exploration strategies may fail to escape local optima.

3.4. On-policy vs Off-policy Learning

In this experiment we compare two temporal-difference learning algorithms: SARSA and Q-learning. Both methods learn action-value functions from interaction with the environment but differ in how the update target is computed.

SARSA is an on-policy algorithm that updates values using the action actually selected by the current policy, as shown in Eq. 1.

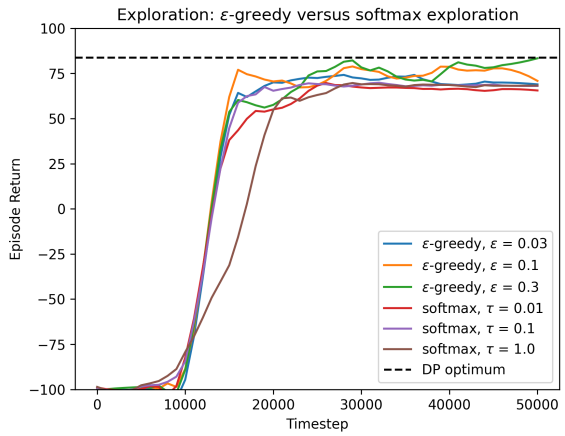


Figure 1. Learning curves for ε-greedy and softmax exploration strategies.

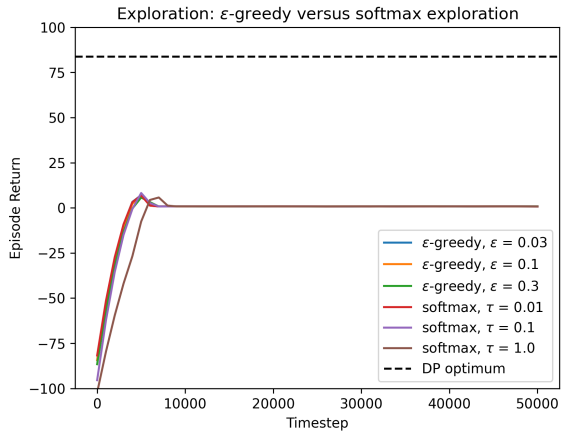


Figure 2. Learning curves with an additional nearby goal.

In contrast, Q-learning is an off-policy method that updates toward the greedy value of the next state, as shown in Eq. 2.

Experimental setup Both algorithms were evaluated using an ε-greedy exploration policy with $\varepsilon = 0.1$ and discount factor $\gamma = 1.0$. Learning curves were generated over $50\,001$ timesteps and averaged across 20 repetitions. Every 1000 timesteps the greedy policy was evaluated and the mean episode return was recorded.

Learning rates $\alpha \in \{0.03, 0.1, 0.3\}$ were tested for both algorithms.

Results Figure 3 shows the resulting learning curves. Higher learning rates generally lead to faster learning but also introduce more variability.

Q-learning tends to achieve slightly higher returns but shows stronger fluctuations during training. This occurs because the update assumes greedy behavior in the future and therefore may exploit trajectories that are sensitive to stochastic wind disturbances.

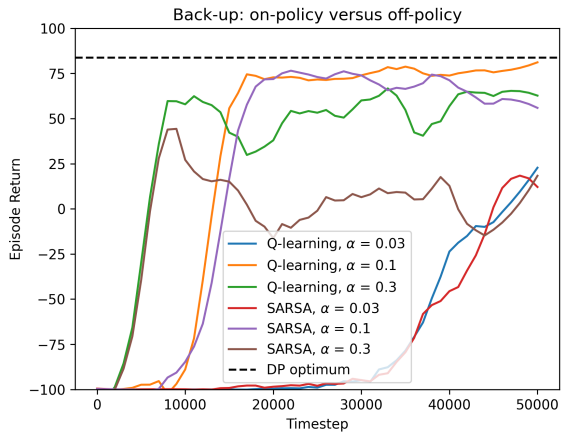


Figure 3. Comparison between SARSA (on-policy) and Q-learning (off-policy) for different learning rates α. Curves are averaged across 20 repetitions.

SARSA, on the other hand, learns more conservative policies because updates are based on the actions actually taken under the exploration policy. As a result, it accounts for the effects of exploratory actions during learning and often avoids risky trajectories near strong wind columns.

3.5. Backup Depth

In this experiment we study how the backup depth affects learning performance. In the one-step case, the update corresponds to the standard Q-learning target defined in Eq. 2. In n-step Q-learning, the update incorporates the rewards collected over the next n timesteps before bootstrapping from the value function:

$$G_t^{(n)} = \sum_{i=0}^{n-1} \gamma^i r_{t+i+1} + \gamma^n \max_a Q(s_{t+n}, a). \qquad (6)$$

For $n = 1$ this reduces to the standard one-step target, while using the full episode return corresponds to Monte Carlo learning.

Experimental setup We evaluated n-step Q-learning with $n \in \{1, 3, 10\}$ and compared it to Monte Carlo learning using ε-greedy exploration with $\varepsilon = 0.05$, learning rate $\alpha = 0.1$, and discount factor $\gamma = 1.0$. Learning curves were averaged across 20 repetitions over $50\,001$ timesteps and evaluated every 1000 steps.

Results Figure 4 shows the learning curves. The 3-step method learns fastest and achieves the best performance, while the one-step method converges more slowly. The

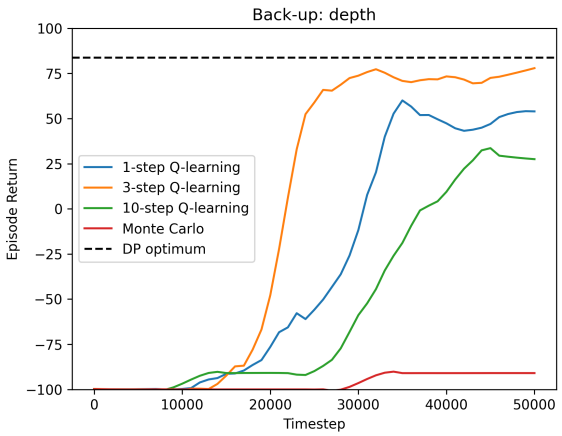


Figure 4. Learning curves for different backup depths.

10-step method is slower and more variable, and Monte Carlo performs worst because updates depend on complete episode returns.

These results illustrate the bias–variance trade-off in temporal-difference learning: short backups rely more on bootstrapping, while longer backups reduce bias but increase variance. In this environment an intermediate backup depth provides the best balance.

A small difference is observed between regular Q-learning and the n-step implementation with $n = 1$. Regular Q-learning performs updates online after each transition, whereas the n-step implementation updates after collecting trajectory segments, leading to slightly different learning behavior.

4. Discussion

Dynamic programming and reinforcement learning differ mainly in the information available to the agent. Dynamic programming assumes full knowledge of transition probabilities and rewards, allowing it to compute optimal policies efficiently and guaranteeing convergence. In contrast, reinforcement learning methods such as Q-learning and SARSA learn directly from interaction with the environment without requiring a model, although they typically require more experience to converge.

Exploration strongly influences learning performance. In our experiments, ε-greedy exploration with $\varepsilon = 0.1$ provided a good balance between exploration and exploitation. Softmax exploration produces smoother action probabilities but requires careful tuning of the temperature parameter. More advanced exploration strategies, such as optimistic initialization or upper-confidence bound methods, could further improve exploration by encouraging systematic investigation of uncertain states.

The reward structure also affects exploration behavior. With a step reward of -1, the agent is encouraged to reach the goal quickly. If the default reward is changed to 0, trajectories appear equally valuable until the goal is reached. In this case the discount factor becomes important: with $\gamma = 1.0$ all successful trajectories yield the same return, whereas with $\gamma = 0.99$ shorter paths remain preferable due to discounting.

The multiple-goal experiment highlights a limitation of simple exploration strategies. When a nearby suboptimal reward exists, the agent may converge to this local optimum instead of discovering a more valuable distant goal, demonstrating the difficulty of escaping deceptive reward structures.

The comparison between SARSA and Q-learning illustrates the difference between on-policy and off-policy learning. Q-learning updates assuming greedy future behavior and may therefore learn policies that exploit risky trajectories in stochastic environments. SARSA instead updates using the action actually taken under the current policy and thus learns safer strategies. Although n-step Q-learning incorporates multiple rewards before bootstrapping, it still uses the greedy value in the final update and therefore remains an off-policy method.

The backup depth experiments illustrate the bias–variance trade-off in temporal-difference learning. Short backups rely heavily on bootstrapped estimates, while longer backups incorporate more observed rewards but introduce higher variance. In our experiments an intermediate backup depth achieved the best performance.

Tabular reinforcement learning methods also suffer from the curse of dimensionality: a separate value must be stored for every state–action pair, making them impractical for large or continuous state spaces and motivating the use of function approximation methods.

5. Conclusion

We investigated several tabular reinforcement learning algorithms in the Stochastic Windy Gridworld. Dynamic programming provided an optimal baseline against which model-free methods were evaluated. Our experiments showed that appropriate exploration is essential for efficient learning, with ε-greedy providing a good balance between exploration and exploitation. The comparison between SARSA and Q-learning highlighted the difference between on-policy and off-policy learning in stochastic environments, while the backup depth experiments illustrated the bias–variance trade-off in temporal-difference methods. Overall, the results demonstrate how exploration strategies, update rules, and backup depth influence learning performance in tabular reinforcement learning.

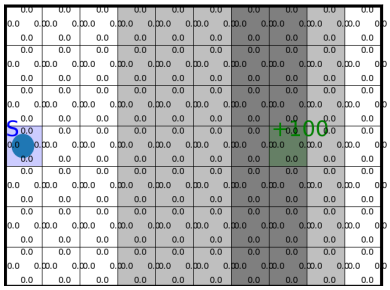
References

Plaat, A. *Deep reinforcement learning*, volume 10. Springer, 2022.

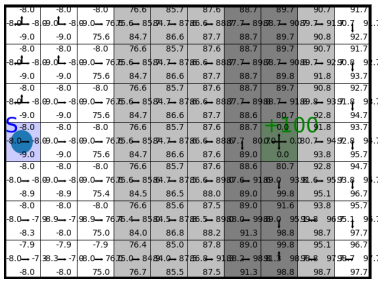
Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

Appendix

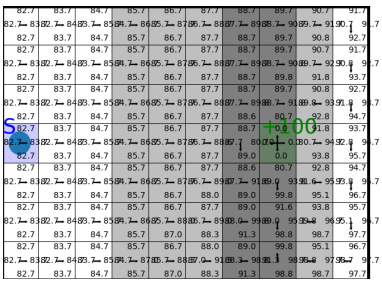
A. Dynamic Programming



(a) Iteration 0: All values initialized to 0.00.



(b) Iteration 8: Reward signal beginning to propagate.



(c) Iteration 17: Final converged values for [7, 3].

Figure 5. Progression of Q-value iteration. The optimal value $V^*(0, 3)$ at convergence is 83.7.